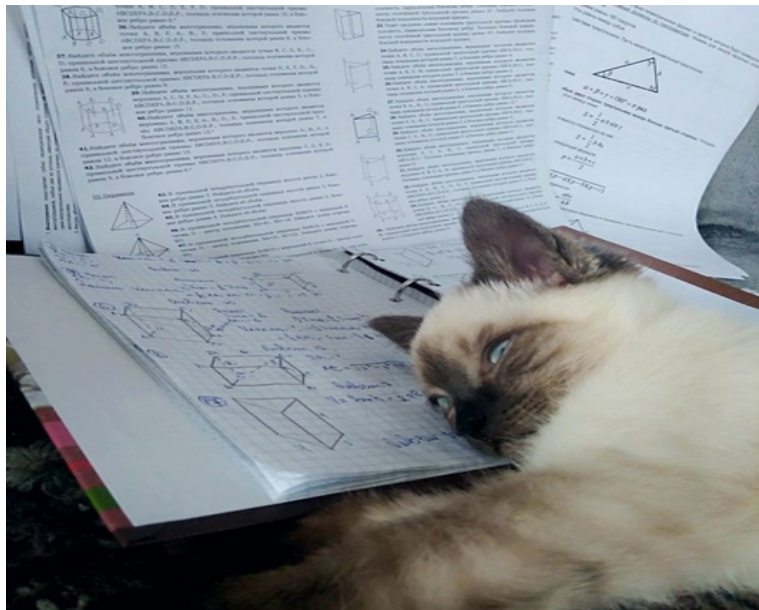




Глава 1. Теория алгоритмов

Ежелев Г. И.

24 июля 2026 г.

Примеры

Понятное дело, что алгоритм, это последовательность действий, которая приводит к вычислению некоторого значения (функции и т.п.). Например: функция от n переменных $(x_1, x_2, \dots, x_n)$, которая на выходе возвращает их в порядке возрастания $(x_{i_1} \leq \dots \leq x_{i_n})$ - это знакомая вам сортировка массива.

Приведем еще несколько примеров алгоритмических задач:

- ▶ Поиск общей подстроки у нескольких входных строк
- ▶ Расчет расстояния до объекта на изображении
- ▶ Локализация, т.е. расчет положения робота на поле
- ▶ Нахождение компонент связности графа
- ▶ Нахождение кратчайшего пути в графе (лабиринте).

Основные понятия

Все алгоритмы будут описаны на псевдокоде. Считается, что умение перенести их на конкретный ЯП (язык программирования) - это ремесло.

Любой алгоритм обычно удовлетворяет 3 основным свойствам:

- ▶ Корректность - при любых входных данных, результат будет соответствовать требованиям (очев)
- ▶ Конечность - нет комбинации входных данных, которые приведут алгоритм к заикливанию
- ▶ Набор команд:
 - Объявление переменных, приравнивание $x:=a$
 - Условия на $<$, $=$, $>$ и циклы с условиями
 - Операторы `goto` и `return`
 - Псевдокод должен в итоге сводиться именно к такому набору операций

Сортировки. Insertion Sort

Основная задача теории алгоритмов - решить задачу оптимальнее очевидного способа.

Очевидный способ отсортировать массив - сортировка пузырьком. Её мы разбирать не будем, это зло.

□ Вход: массив $(x_1, \dots, x_n)$

□ Задача: Отсортировать массив по возрастанию. □ Описание алгоритма:

- ▶ Алгоритм напоминает сортировку колоды игровых карт.
- ▶ Массив делится на две части: отсортированная и еще нет. В начале отсортированная часть - это один элемент x_1
- ▶ Переход $i-1 \rightarrow i$: первые $i-1$ элементов отсортированы. Хотим **вставить** i -ый элемент в отсортированную часть

Алгоритм 1 Insertion Sort

Вход: Массив $A := x_1, \dots, x_n$

```
1: function QUICKSORT(A)
2:   for  $i = 2 \dots n$  do                                ▷ Цикл по длине отсортированной части
3:      $x := A[i]$                                            ▷ Запомнили первый неотсортированный элемент
4:     while  $j \geq 1 \ \& \ x < A[j]$  do                    ▷ Перебираем колоду карт
5:        $A[j+1] = A[j]$                                      ▷ Освобождаем место под X
6:        $j := j-1$ 
7:     end while
8:      $A[j+1] = x$                                          ▷ Вставляем X в освободившееся место
9:   end for
10: end function
```

Сложность алгоритма.

Оптимальность алгоритма можно оценивать через его асимптотическую сложность. Для этого введем несколько определений:

- ▶ Сложность алгоритма - количество его итераций (повторений действий) или количество действий, для расчета результата.
- ▶ Сложность алгоритма сложно рассчитать явно, а вот прикинуть можно через O-нотацию (Big-O)
- ▶ Пусть f и g - функции. $g(n) := O(f(n))$, если для некоторой константы $C > 0$ $g(n) \leq C * f(n)$
- ▶ То есть функция g всегда ограничена сверху функцией f , умноженной на константу.
- ▶ Т.к. время исполнения одной итерации цикла рассчитать сложно (действительно, не очевидно сколько времени займет операция $j = j-1$ и т.п.), то мы можем принять это время за некоторую константу C . А вот количество итераций цикла посчитать обычно проще.
- ▶ Именно поэтому и используется Big-O она позволяет отбросить эту константу и смотреть лишь на "порядок" сложности.

Сложность Insertion Sort.

Корректность алгоритма очевидна, давайте оценим сложность. В данном случае время алгоритма очень сильно зависит от "степени упорядоченности" исходного массива

Лемма 1

□ Лучшее время: $O(n)$

□ Худшее время: $O(n^2)$

▷

- Пусть массив уже отсортирован. Тогда внешний цикл пробежит каждый элемент, а внутренний цикл на первой же итерации получит противоречие с условием $x \leq A[j]$ и не будет сдвигать массив.
- Таким образом получим n итераций $\Rightarrow$ сложность $O(n)$

Лемма 1

- Лучшее время: $O(n)$
- Худшее время: $O(n^2)$
- Рассмотрим самую грубую худшую оценку: пусть внутренний цикл каждый раз будет пробегать "почти всю" отсортированную часть.
- Внешний цикл пробежит весь массив в любом случае. Таким образом на каждую из n итераций внешнего цикла мы получили "почти n " итераций внутреннего.
- Тогда всего $\approx n^2$ итераций $\Rightarrow$ сложность $O(n^2)$
- Наихудший случай соответствует массиву, отсортированному наоборот, т.е. по убыванию.

◁

Чтобы сравнить скорость алгоритмов, достаточно сравнить их Big-O нотации

InsertionSort - очень простой, но не очень быстрый алгоритм. Если скорость роляет, лучше использовать подход ниже.

Продвинутая сортировка. MergeSort

Merge Sort - Сортировка слиянием использует подход - "Разделяй и властвуй". Разделим задачу на подзадачи, которые по-отдельности решать быстрее, чем вместе, а лучше подзадачи разделим на под-подзадачи, и так до тех пор, пока решение не станет тривиальным.

□ Описание алгоритма:

- ▶ Разделим массив пополам и рекурсивно вызовем алгоритм для каждой половинки, когда рекурсия закончится - получим отсортированные половинки
- ▶ Далее их надо слить воедино.
- ▶ Это можно сделать за линейное время: поставим указатели на начало каждой половинки. Пусть это указатели p и q . Также добавим указатель в массиве-результате
- ▶ Если $A[p] \leq A[q]$, то добавим $A[p]$ в $A[r]$: $A[p] := A[r]$, иначе $A[q]$
- ▶ Когда одна половинка закончится, вся оставшаяся будет добавлена после $A[r]$, т.е. в конец массива.

Алгоритм 2 Merge Sort

Вход: Массив $A := x_1, \dots, x_n$

Выход: Отсортированный массив R

```
1: function MERGE_SORT( $A$ )
2:   MERGE_SORT( $(x_1, \dots, x_{\lfloor \frac{n}{2} \rfloor})$ )
3:   MERGE_SORT( $(x_{\lfloor \frac{n}{2} \rfloor + 1}, \dots, x_n)$ )
4:   MERGE( $(x_1, \dots, x_{\lfloor \frac{n}{2} \rfloor})$ ,  $(x_{\lfloor \frac{n}{2} \rfloor + 1}, \dots, x_n)$ )
5: end function
6: —————
7: function MERGE( $A_0, A_1$ )
8:   Пусть  $p_0, q_0$  — указатели на начала половинок
9:   Пусть  $p_1, q_1$  — указатели на концы половинок
10:  Пусть  $p := p_0, q := q_0, r = 0$  — текущий элемент
```

N.B. Естественно рекурсию надо ограничить, самый очевидный вариант - разбитые половинки состоят из одного элемента, тогда их можно вернуть сразу, без процедуры merge.

Отметим, что алгоритм сразу меняет массив R, поэтому нету return.

```
11:  while  $p < p_1 \& q < q_1$  do
12:      if  $A_0[p] \leq A_1[q]$  then
13:           $R[r] := A_0[p]$ 
14:           $p++$ 
15:           $r++$ 
16:      else
17:           $R[r] := A_1[q]$ 
18:           $q++$ 
19:           $r++$ 
20:      end if
21:  end while
22: end function
```

Сложность Merge Sort.

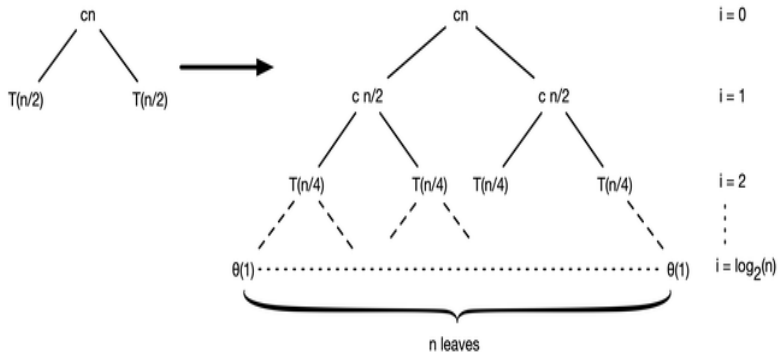
Лемма 2

Сложность Merge Sort $O(n * \log_2(n))$



- Алгоритм действует по рекурсии. Посчитаем количество действий на каждом уровне рекурсии.
- Вообще говоря, функция merge_sort просто делит массив, никаких действий она не выполняет. Значит она делает почти нулевой вклад в итоговую сложность алгоритма.
- На первом (исходном) уровне операция Merge пробегается циклом по каждому элементу каждой половинки ровно один раз. Т.е. ровно n итераций.
- На втором уровне длина каждой половинки $\frac{n}{4}$, таких половинок тоже 4
- На уровне рекурсии i получим 2^i половинок, каждая длины $\frac{n}{2^i}$.
- Всего $2^i * \frac{n}{2^i} = n$ операций на каждом уровне.
- А сколько всего уровней?
- Массив при рекурсивных вызовах разбивается на неполное бинарное дерево, его высота $\log_2 n$ (см. картинку ниже)





● На самом деле можно доказать, что алгоритм сортировки не может гарантированно отсортировать массив за менее чем $n * \log_2 n$ сравнений элементов, ему просто не хватит информации. Более быстрые алгоритмы используют еще что-то кроме сравнений $<, >$.

● Если хотите еще поизучать тему, то почитайте про QuickSort.

Задание: Реализуйте один из двух алгоритмов на C++ в любой ide.